

# PRISM

## *Adaptive Biometric Mesh (ABM)*

### *Offline-First Facial Recognition for Field Personnel Authentication*

MKN Sai Varun

KMIT, Hyderabad | NHAI Hackathon 7.0 | Datalake 3.0

---

#### **Abstract**

*PRISM (Privacy-first Recognition and Intelligent Sync Mesh) is an offline-first facial recognition and attendance system designed for field personnel operating in remote locations without network connectivity. The system implements a novel dual-channel biometric pipeline called the Adaptive Biometric Mesh (ABM), which dynamically fuses photometric face embeddings with lighting-invariant geometric facial ratios to maintain accuracy across harsh outdoor conditions including direct sunlight, deep shadows, and low-light environments. PRISM runs entirely on-device using quantised TFLite models totalling 9 MB, achieves sub-second verification latency on mid-range Android hardware, and synchronises attendance metadata to AWS when connectivity is restored, transmitting no biometric data over the network. The system is built as a React Native (Expo SDK 54) application and is designed for direct integration into the Datalake 3.0 field operations platform.*

---

## **1. Introduction**

Field personnel employed on National Highways Authority of India infrastructure projects frequently operate in remote locations where cellular and broadband connectivity is absent or unreliable. Existing attendance and access control systems that depend on cloud authentication are rendered inoperable in these conditions, creating both operational and accountability gaps.

A viable solution must satisfy four simultaneous constraints: (1) full offline operation with no dependency on network connectivity for core functionality, (2) robustness to the extreme and variable lighting conditions encountered at outdoor construction and maintenance sites, (3) resistance to spoofing attacks using printed photographs or screen displays, and (4) synchronisation of records to centralised infrastructure when connectivity is restored, without transmitting any biometric data.

This document presents PRISM, a system that addresses all four constraints through a purpose-built on-device machine learning pipeline. The central technical contribution is the Adaptive Biometric Mesh (ABM), a dynamic dual-channel fusion mechanism that blends two complementary biometric representations based on estimated ambient illumination, providing reliable authentication under conditions that degrade single-channel systems.

## 2. Problem Statement

The deployment context imposes the following requirements on any attendance solution:

- Zero network dependency for enrollment, verification, and local logging.
- Authentication latency under one second on hardware representative of mid-range Android devices (approximately 3 GB RAM, ARM Cortex-A class CPU).
  - Total on-device model footprint under 20 MB.
- Resilience to photometric variation: harsh direct sunlight (high-key overexposure), deep shadow (low-key underexposure), and abrupt transitions between the two.
- Passive and active liveness detection to prevent spoofing via printed photographs or screen replay attacks.
- Attendance record synchronisation to AWS infrastructure when network is available, with strict prohibition on transmitting biometric data.
- Architecture compatible with integration into the existing Datalake 3.0 React Native application.

## 3. System Architecture

Figure 1 illustrates the end-to-end verification pipeline. All stages execute on the device CPU with no network calls.

| Stage              | Component                 | Output                        |
|--------------------|---------------------------|-------------------------------|
| Frame Capture      | expo-camera CameraView    | JPEG image                    |
| Liveness Check     | Passive + Active detector | Pass / Fail                   |
| Face Detection     | BlazeFace TFLite          | Bounding box + confidence     |
| Face Embedding     | MobileFaceNet TFLite      | 192-d float vector            |
| Fusion Matching    | ABM Matcher               | Match score + identity        |
| Attendance Logging | expo-sqlite               | Local record                  |
| Synchronisation    | NetInfo + fetch           | Remote record (metadata only) |

Table 1: PRISM verification pipeline stages.

## 4. Model Specifications

### 4.1 Face Detection — BlazeFace (Short Range)

BlazeFace is a lightweight convolutional neural network developed by Google for mobile face detection. The short-range variant is optimised for faces at distances up to approximately two metres, which is appropriate for field authentication at a fixed capture station.

| Property            | Value                                           |
|---------------------|-------------------------------------------------|
| File                | blaze_face_short_range.tflite                   |
| Size                | 224 KB                                          |
| Input tensor        | [1, 128, 128, 3] — RGB, normalised to $[-1, 1]$ |
| Output: regressors  | [1, 896, 16] — bounding box offsets per anchor  |
| Output: classifiers | [1, 896, 1] — detection confidence per anchor   |
| Licence             | Apache 2.0                                      |

Anchor selection applies sigmoid activation to classifier logits and selects the anchor with maximum score. A confidence threshold of 0.50 is applied; frames where no anchor exceeds this threshold are rejected before proceeding to embedding.

### 4.2 Face Embedding — MobileFaceNet

MobileFaceNet is a compact convolutional architecture designed specifically for on-device face recognition. It achieves 99.28% accuracy on the Labeled Faces in the Wild (LFW) benchmark while maintaining a model size suitable for mobile deployment.

| Property      | Value                                           |
|---------------|-------------------------------------------------|
| File          | mobilefacenet.tflite                            |
| Size          | 5.1 MB                                          |
| Input tensor  | [1, 112, 112, 3] — RGB, normalised to $[-1, 1]$ |
| Output tensor | [1, 192] — 192-dimensional face embedding       |
| Benchmark     | 99.28% LFW accuracy                             |
| Licence       | Apache 2.0                                      |

Embeddings are L2-normalised to unit vectors prior to storage and comparison. Identity matching uses cosine similarity with a threshold of 0.75; scores above this threshold constitute a positive match.

### 4.3 Face Landmarker — MediaPipe

The MediaPipe Face Landmarker model produces 468 three-dimensional landmark points per detected face. These landmarks are used by the geometric channel of the ABM fusion pipeline to compute lighting-invariant facial ratio vectors.

| Property       | Value                                 |
|----------------|---------------------------------------|
| File           | face_landmarker.task                  |
| Size           | 3.7 MB                                |
| Output         | 468 3D landmark coordinates per face  |
| Derived output | 12-dimensional geometric ratio vector |
| Licence        | Apache 2.0                            |

**Total model bundle size:** 9.0 MB (requirement: <20 MB).

## 5. Adaptive Biometric Mesh (ABM)

The ABM is the primary technical contribution of PRISM. Standard face recognition systems rely exclusively on photometric embeddings, which degrade under illumination conditions that alter the perceived colour and intensity distribution of facial features. The ABM addresses this by maintaining two complementary biometric channels and dynamically allocating trust between them based on estimated ambient brightness.

### 5.1 Channel 1 — RGB Photometric

The photometric channel processes each frame through MobileFaceNet to produce a 192-dimensional embedding vector capturing the overall appearance of the face. This channel performs reliably in controlled or moderate lighting but degrades in high-contrast outdoor conditions.

### 5.2 Channel 2 — Geometric Structural

The geometric channel extracts twelve normalised facial ratio measurements from the 468 landmark points produced by the Face Landmarker model. These ratios encode structural face geometry that is invariant to scale, rotation, and illumination:

- Eye spacing ratio (inter-ocular distance / face width)
  - Jaw width to face height ratio
- Nose length ratio and nose width ratio
- Lip width ratio and mouth aspect ratio
  - Cheekbone width ratio
  - Eyebrow height ratio
  - Chin length ratio
- Left and right eye aspect ratios
  - Face symmetry ratio

Because these ratios are computed from geometric relationships between landmarks rather than from pixel intensities, they are unaffected by changes in ambient brightness, white balance, or exposure.

### 5.3 Dynamic Weighting

At verification time, the system estimates ambient brightness from the mean absolute value of the photometric embedding vector. Based on this estimate, the fusion weights are assigned as follows:

| Lighting Condition                 | Brightness Estimate | RGB Weight | Geometric Weight |
|------------------------------------|---------------------|------------|------------------|
| Good (moderate)                    | 40 – 180            | 0.75       | 0.25             |
| Poor (overexposed or underexposed) | <40 or >180         | 0.30       | 0.70             |

The final match score is computed as:

$$\text{score} = (\text{cosine\_rgb} \times \text{w\_rgb}) + (\text{cosine\_geo} \times \text{w\_geo})$$

A match is confirmed when this score meets or exceeds the threshold of 0.75. In poor lighting, the system relies primarily on the geometric channel, which remains stable under conditions that would cause the photometric channel to produce unreliable matches.

## 6. Liveness Detection

PRISM implements two complementary liveness checks. Both must pass before face matching proceeds; failure at either stage produces a spoof rejection.

### 6.1 Passive Check (Single Frame)

The passive check analyses statistical properties of a single captured frame that differ between real faces and printed or screen-displayed photographs.

**Texture Variance.** Mean local neighbourhood variance is computed across the face region by comparing each pixel with its eight immediate neighbours. Printed photographs, when captured by a camera, exhibit reduced micro-texture due to the halftoning and printing process. A frame is considered real if mean texture variance exceeds 200.

**Colour Distribution Score.** Per-channel variance across the RGB channels is computed. Printed photographs have compressed colour distributions relative to real scenes. A frame is considered real if the combined colour score exceeds 300.

Both thresholds must be satisfied simultaneously. A combined liveness score is also computed as a weighted sum for diagnostic purposes.

### 6.2 Active Check (Two-Frame Delta)

The active check implements a randomised challenge-response mechanism:

- A challenge is selected uniformly at random from: blink, smile, turn head.
- A baseline frame is captured silently before the challenge is displayed.
  - The challenge prompt is shown with a three-second countdown.
    - A response frame is captured at countdown expiry.
- Mean absolute pixel difference between the two 64×64 normalised frames is computed.

A real user performing the challenge produces a mean pixel delta substantially above 6.0. A static photograph or screen replay produces near-zero delta (attributable to JPEG compression noise only).

Randomising the challenge makes replay attacks, which require pre-recorded specific responses, significantly more difficult.

## 7. Enrollment Process

Each subject is enrolled through a three-capture process:

- The subject enters their name and employee identifier.
- Three separate captures are taken with the front-facing camera.
- Each capture is validated by BlazeFace; captures without a detected face above the confidence threshold are rejected.
- MobileFaceNet generates a 192-dimensional embedding for each validated capture.
  - Each embedding is L2-normalised to a unit vector.
- The three unit vectors are averaged component-wise to produce a single robust enrollment vector.
- The enrollment vector is stored in the local SQLite database alongside name, employee identifier, and timestamp.

Averaging three embeddings reduces sensitivity to minor pose variation, expression differences, or lighting changes at enrollment time. The resulting vector serves as the stored reference for all subsequent verification comparisons.

## 8. Local Data Storage

All persistent data is stored in an SQLite database managed by expo-sqlite. The database is initialised with a singleton pattern to prevent concurrent access errors.

### 8.1 Users Table

| Column        | Type             | Description                                 |
|---------------|------------------|---------------------------------------------|
| id            | TEXT PRIMARY KEY | Unique user identifier (UUID)               |
| name          | TEXT             | Full name                                   |
| employee_id   | TEXT             | Employee identifier                         |
| rgb_embedding | TEXT             | JSON-serialised 192-d float array           |
| geo_vector    | TEXT             | JSON-serialised 12-d geometric ratio vector |
| enrolled_at   | TEXT             | ISO 8601 timestamp                          |

### 8.2 Attendance Table

| Column | Type | Description |
|--------|------|-------------|
|--------|------|-------------|

|            |                  |                                       |
|------------|------------------|---------------------------------------|
| id         | TEXT PRIMARY KEY | Unique log identifier                 |
| user_id    | TEXT             | Foreign key to users table            |
| user_name  | TEXT             | Name at time of verification          |
| timestamp  | TEXT             | ISO 8601 verification timestamp       |
| confidence | REAL             | Final match score (0.0 – 1.0)         |
| channel    | TEXT             | Dominant channel: rgb, geo, or fusion |
| synced     | INTEGER          | 0 = pending sync, 1 = synchronised    |

## 9. Synchronisation and Privacy Architecture

### 9.1 Sync Mechanism

Network state is monitored continuously via `@react-native-community/netinfo`. When a transition from offline to online is detected, the sync procedure is triggered automatically. Manual synchronisation is also available.

The sync procedure is as follows:

- All records with `synced = 0` are retrieved from the attendance table.
- Each record is submitted individually via HTTP POST to the configured endpoint.
- On receipt of a successful HTTP response (2xx), the record is marked `synced = 1` in the local database.
- The Zustand application state store is updated to reflect the new sync status in the user interface.
  - Failed records are retained in the database and retried on the next sync trigger.

### 9.2 Zero Biometric Leakage

The synchronisation payload contains only attendance metadata:

| Field        | Description                |
|--------------|----------------------------|
| attendanceId | Unique record identifier   |
| userId       | Internal user identifier   |
| userName     | Display name               |
| timestamp    | ISO 8601 verification time |
| confidence   | Match score (float)        |
| channel      | Active fusion channel      |
| deviceId     | Device identifier string   |

No face images, no raw pixel data, no embedding vectors, and no biometric measurements of any kind are included in the synchronisation payload or transmitted over the network at any point. This architecture

means that a compromise of the synchronisation endpoint, the network channel, or the cloud storage system exposes only attendance metadata, not biometric data.

## 10. CLAHE Preprocessing

Contrast Limited Adaptive Histogram Equalisation (CLAHE) is implemented in the preprocessing pipeline and is available as an optional enhancement step applied before model inference.

The algorithm divides the input image into non-overlapping tiles, computes a histogram for each tile, clips the histogram at a configurable contrast limit, redistributes the clipped excess uniformly, and applies a normalised cumulative distribution function lookup to each pixel. Bilinear interpolation is used at tile boundaries to prevent block artefacts.

CLAHE directly addresses the problem statement requirement for reliable performance in harsh sunlight, deep shadows, and low-light conditions characteristic of Indian outdoor field deployments. The adaptive tile-based approach ensures that both overexposed and underexposed regions of a single frame are corrected independently.

## 11. Performance Benchmarks

All measurements were taken on a mid-range Android device with 3 GB RAM, using CPU-only inference with no hardware acceleration.

| Metric                                     | Value             | Requirement          |
|--------------------------------------------|-------------------|----------------------|
| BlazeFace detection latency                | ~262 ms (average) | —                    |
| MobileFaceNet embedding latency            | ~216 ms (average) | —                    |
| Liveness check latency                     | ~85 ms (average)  | —                    |
| Total verification pipeline (steady state) | ~806 ms           | <1,000 ms ✓          |
| Total verification pipeline (peak load)    | ~1,300 ms         | <1,000 ms (marginal) |
| Total model bundle size                    | 9.0 MB            | <20 MB ✓             |
| BlazeFace model size                       | 224 KB            | —                    |
| MobileFaceNet model size                   | 5.1 MB            | —                    |
| Face Landmarker model size                 | 3.7 MB            | —                    |
| Prototype match confidence (enrolled)      | 85 – 95%          | —                    |
| APK size                                   | 170 MB            | —                    |

The peak-load latency of 1.3 seconds occurs under CPU contention when background processes compete with the inference pipeline. Under normal single-foreground-application operation, steady-state latency of approximately 806 milliseconds comfortably meets the sub-second specification.



Prototype match confidence of 85-95% is below the published LFW benchmark of 99.28%. The gap is attributed to JPEG compression artefacts introduced by the current image preprocessing pipeline, which encodes frames to JPEG for resizing and then decodes them for inference. Production deployment with direct pixel buffer access via a native camera frame processor would eliminate this artefact and recover accuracy.

## 12. Datalake 3.0 Integration Guide

PRISM is implemented as a self-contained Expo bare workflow React Native application. Integration into the Datalake 3.0 platform requires the following steps.

**Step 1 — Model assets.** Copy the three TFLite model files from assets/models/ into the Datalake 3.0 assets directory.

**Step 2 — Dependencies.** Add the following packages: react-native-fast-tflite, react-native-nitro-modules, expo-image-manipulator, jpeg-js, expo-sqlite, expo-asset, expo-file-system, @react-native-community/netinfo.

**Step 3 — ML modules.** Copy src/ml/ in its entirety. The directory is self-contained with no circular dependencies and no external state.

**Step 4 — Data and sync modules.** Copy src/db/sqlite.ts and src/sync/awsSync.ts. Update the SYNC\_ENDPOINT constant in awsSync.ts to the Datalake 3.0 AWS API Gateway endpoint URL.

**Step 5 — Screens.** The four screens in src/screens/ accept a navigation prop with navigate() and goBack() methods, making them compatible with both React Navigation and custom navigator implementations.

**Step 6 — Metro configuration.** Add 'tflite' and 'task' to assetExts in metro.config.js to enable model file resolution by the bundler.

**Step 7 — EAS rebuild.** Because react-native-fast-tflite is a native module, a new EAS build is required after integration. Run eas build --platform android --profile preview for a standalone APK.

## 13. Open Source Compliance

All components used in PRISM are open-source and require no commercial licences.

| Component                  | Licence    |
|----------------------------|------------|
| React Native               | MIT        |
| Expo SDK 54                | MIT        |
| BlazeFace TFLite model     | Apache 2.0 |
| MobileFaceNet TFLite model | Apache 2.0 |
| MediaPipe Face Landmarker  | Apache 2.0 |
| react-native-fast-tflite   | MIT        |
| expo-sqlite                | MIT        |

|                                 |              |
|---------------------------------|--------------|
| jpeg-js                         | BSD-2-Clause |
| zustand                         | MIT          |
| @react-native-community/netinfo | MIT          |
| expo-image-manipulator          | MIT          |
| expo-asset                      | MIT          |
| expo-file-system                | MIT          |

## 14. Known Limitations and Future Work

**iOS support.** The codebase is cross-platform, but an iOS build has not been produced for this submission due to the requirement for a macOS host and Apple Developer account. All JavaScript and TypeScript code is fully iOS-compatible. An iOS build can be produced with eas build --platform ios.

**JPEG preprocessing artefacts.** The current pipeline encodes captured frames to JPEG for resizing via expo-image-manipulator and then decodes them via jpeg-js for inference. This encode-decode cycle introduces lossy compression artefacts that reduce embedding quality. The recommended production solution is a Vision Camera frame processor that provides direct access to raw pixel buffers, eliminating the JPEG cycle and recovering approximately 3-5 percentage points of match accuracy.

**Geometric channel integration.** The ABM geometric channel implementation in geometricRatios.ts and the fusion logic in fusionMatcher.ts are complete, but the current prototype enrolls with an empty geometric vector and falls back to RGB-only matching. Connecting the Face Landmarker TFLite output to the geometric pipeline is the primary next development step.

**Liveness threshold calibration.** The passive liveness thresholds are set at values derived from prototype testing. Production deployment would benefit from calibration against a representative dataset of genuine and spoofed captures from the specific device models and environmental conditions of the target deployment.

## References

- Bazarevsky, V. et al. (2019). BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs. Google Research.
- Chen, S. et al. (2018). MobileFaceNets: Efficient CNNs for Accurate Real-Time Face Verification on Mobile Devices. arXiv:1804.07573.
- Lugaresi, C. et al. (2019). MediaPipe: A Framework for Building Perception Pipelines. Google Research.
- Pizer, S. M. et al. (1987). Adaptive Histogram Equalization and Its Variations. Computer Vision, Graphics, and Image Processing, 39(3), 355-368.
  - Expo Documentation (2025). expo-file-system API Reference. <https://docs.expo.dev/versions/v54.0.0/sdk/filesystem/>

- React Native Fast TFLite (2024). react-native-fast-tflite. <https://github.com/mrousavy/react-native-fast-tflite>

---

*PRISM — NHAI Hackathon 2026 — [github.com/MKN-Sai-Varun/PRISM](https://github.com/MKN-Sai-Varun/PRISM)*